Laboratory Assignment 5

Implementation of a Radix-4 Booth Multiplier IP with an AXI4 Interface

Shahan Antabian / Ayush Mohpatra
High-Level Digital Design

May 26, 2026

Contents

1 Objective **3**

2 Hardware: AXI4-Lite interface **3**
2.1 Instantiation 3

3 Software Application **11**
3.1 Booth ISR 11
3.2 Exception Handler 12
3.3 Main Application 13

4 Results **15**

5 Resource Utilization **16**

6 Timing Analysis Report Summary **16**

1 Objective

The aim of this assignment is to take the 16-bit signed Booth multiplier designed in Lab 4 and integrate it as an AXI4-Lite peripheral in a Cyclone V SoC system. The peripheral is mapped into the lightweight HPS-to-FPGA bridge, and software running on the ARM Cortex-A9 processor drives the multiplier and reads its result through interrupts.

2 Hardware: AXI4-Lite interface

2.1 Instantiation

- `data_a[15:0]` is stored in `slv_reg0`
- `data_a[15:0]` is stored in `slv_reg1`
- `result[31:0]` is mapped to `slv_reg2`,
- `start` is stored in `slv_reg3(0)`
- `busy` is mapped in `slv_reg3(1)`,
- `ack` is stored in `slv_reg3(2)`
- `irq` is connected using an external port.

Table 1: AXI4-Lite register map.

Address	Register	Use
0xFF200000	slv_reg0	`data_a` (multiplicand, write)
0xFF200004	slv_reg1	`data_b` (multiplier, write)
0xFF200008	slv_reg2	32-bit result (read)
0xFF20000C	slv_reg3	bit 0: start (W)
		bit 1: busy (R)
		bit 2: ack (W)

The following changes has been done.

- An external port `irq` is added to the entity.
- A component declaration for `booth_mult` (the module designed in Lab 4) is added to the architecture with the required signals as it is mentioned in table 1 .
- Internal signals `signal_busy` and `signal_result` are declared to connect the multiplier outputs to the registers 2 and 3.
- The Booth multiplier is instantiated and its ports are wired to the appropriate slv_reg slices.

The complete file is listed below.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5
6  entity axi4_lite_booth is
7     generic (
8         -- Users to add parameters here
9
10
11        -- User parameters ends
12        -- Do not modify the parameters beyond this line
```

```
13
14
15          -- Width of S_AXI data bus
16          C_S_AXI_DATA_WIDTH  : integer    := 32;
17          -- Width of S_AXI address bus
18          C_S_AXI_ADDR_WIDTH  : integer    := 4
19      );
20      port (
21          -- Users to add ports here
22          -- automatic irq generated from booth multiplier stored in an external
                register
23              irq      : out std_logic;
24          -- User ports ends
25          -- Do not modify the ports beyond this line
26
27
28          -- Global Clock Signal
29          clk          : in std_logic;
30          -- Global Reset Signal. This Signal is Active LOW
31          reset_n      : in std_logic;
32          -- Write address (issued by master, acceped by Slave)
33          axs_s0_AXI_AWADDR    : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
34          -- Write channel Protection type. This signal indicates the
35              -- privilege and security level of the transaction, and whether
36              -- the transaction is a data access or an instruction access.
37          axs_s0_AXI_AWPROT    : in std_logic_vector(2 downto 0);
38          -- Write address valid. This signal indicates that the master signaling
39              -- valid write address and control information.
40          axs_s0_AXI_AWVALID   : in std_logic;
41          -- Write address ready. This signal indicates that the slave is ready
42              -- to accept an address and associated control signals.
43          axs_s0_AXI_AWREADY   : out std_logic;
44          -- Write data (issued by master, acceped by Slave)
45          axs_s0_AXI_WDATA     : in std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
46          -- Write strobes. This signal indicates which byte lanes hold
47              -- valid data. There is one write strobe bit for each eight
48              -- bits of the write data bus.
49          axs_s0_AXI_WSTRB     : in std_logic_vector((C_S_AXI_DATA_WIDTH/8)-1 downto
                0);
50          -- Write valid. This signal indicates that valid write
51              -- data and strobes are available.
52          axs_s0_AXI_WVALID    : in std_logic;
53          -- Write ready. This signal indicates that the slave
54              -- can accept the write data.
55          axs_s0_AXI_WREADY    : out std_logic;
56          -- Write response. This signal indicates the status
57              -- of the write transaction.
58          axs_s0_AXI_BRESP     : out std_logic_vector(1 downto 0);
59          -- Write response valid. This signal indicates that the channel
60              -- is signaling a valid write response.
61          axs_s0_AXI_BVALID    : out std_logic;
62          -- Response ready. This signal indicates that the master
63              -- can accept a write response.
64          axs_s0_AXI_BREADY    : in std_logic;
65          -- Read address (issued by master, acceped by Slave)
66          axs_s0_AXI_ARADDR    : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
67          -- Protection type. This signal indicates the privilege
68              -- and security level of the transaction, and whether the
69              -- transaction is a data access or an instruction access.
70          axs_s0_AXI_ARPROT    : in std_logic_vector(2 downto 0);
71          -- Read address valid. This signal indicates that the channel
72              -- is signaling valid read address and control information.
73          axs_s0_AXI_ARVALID   : in std_logic;
74          -- Read address ready. This signal indicates that the slave is
```

```
            -- ready to accept an address and associated control signals.
        axs_s0_AXI_ARREADY  : out std_logic;
        -- Read data (issued by slave)
        axs_s0_AXI_RDATA    : out std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
        -- Read response. This signal indicates the status of the
            -- read transfer.
        axs_s0_AXI_RRESP    : out std_logic_vector(1 downto 0);
        -- Read valid. This signal indicates that the channel is
            -- signaling the required read data.
        axs_s0_AXI_RVALID   : out std_logic;
        -- Read ready. This signal indicates that the master can
            -- accept the read data and response information.
        axs_s0_AXI_RREADY   : in std_logic
    );
end axi4_lite_booth;


architecture arch_imp of axi4_lite_booth is

    -- AXI4LITE signals
    signal axi_awaddr   : std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
    signal axi_awready  : std_logic;
    signal axi_wready   : std_logic;
    signal axi_bresp    : std_logic_vector(1 downto 0);
    signal axi_bvalid   : std_logic;
    signal axi_araddr   : std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
    signal axi_arready  : std_logic;
    signal axi_rdata    : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal axi_rresp    : std_logic_vector(1 downto 0);
    signal axi_rvalid   : std_logic;


    -- Example-specific design signals
    -- local parameter for addressing 32 bit / 64 bit C_S_AXI_DATA_WIDTH
    -- ADDR_LSB is used for addressing 32/64 bit registers/memories
    -- ADDR_LSB = 2 for 32 bits (n downto 2)
    -- ADDR_LSB = 3 for 64 bits (n downto 3)
    constant ADDR_LSB  : integer := (C_S_AXI_DATA_WIDTH/32)+ 1;
    constant OPT_MEM_ADDR_BITS : integer := 1;
    ------------------------------------------------
    ---- Signals for user logic register space example
    ------------------------------------------------
    ---- Number of Slave Registers 4
    signal slv_reg0 :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal slv_reg1 :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal slv_reg2 :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal slv_reg3 :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal slv_reg_rden : std_logic;
    signal slv_reg_wren : std_logic;
    signal reg_data_out :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal byte_index   : integer;
    signal aw_en    : std_logic;

    ------------------------------------------------
    ---- Booth multiplier ports declarations (components and signals)
    ---- to be included here
    ------------------------------------------------



    component booth_mult is
    port(
        clk: in std_logic;                      -- clock input, active by rising
            edge
```

```
138         resetn: in std_logic;                    -- synchronous reset input, active
                low
139         start: in std_logic;
140         busy: out std_logic;
141         irq: out std_logic;
142         ack: in std_logic;
143         data_a: in std_logic_vector(15 downto 0);
144         data_b: in std_logic_vector(15 downto 0);
145         result: out std_logic_vector(31 downto 0)
146     );
147     end component;
148 
149     signal signal_busy: std_logic;
150     signal signal_result: std_logic_vector(31 downto 0);
151 
152 
153 
154 
155 begin
156     -- I/O Connections assignments
157 
158 
159     axs_s0_AXI_AWREADY  <= axi_awready;
160     axs_s0_AXI_WREADY   <= axi_wready;
161     axs_s0_AXI_BRESP    <= axi_bresp;
162     axs_s0_AXI_BVALID   <= axi_bvalid;
163     axs_s0_AXI_ARREADY  <= axi_arready;
164     axs_s0_AXI_RDATA    <= axi_rdata;
165     axs_s0_AXI_RRESP    <= axi_rresp;
166     axs_s0_AXI_RVALID   <= axi_rvalid;
167     -- Implement axi_awready generation
168     -- axi_awready is asserted for one S_AXI_ACLK clock cycle when both
169     -- S_AXI_AWVALID and S_AXI_WVALID are asserted. axi_awready is
170     -- de-asserted when reset is low.
171 
172 
173     process (clk)
174     begin
175       if rising_edge(clk) then
176         if reset_n = '0' then
177           axi_awready <= '0';
178           aw_en <= '1';
179         else
180           if (axi_awready = '0' and axs_s0_AXI_AWVALID = '1' and axs_s0_AXI_WVALID
                  = '1' and aw_en = '1') then
181             -- slave is ready to accept write address when
182             -- there is a valid write address and write data
183             -- on the write address and data bus. This design
184             -- expects no outstanding transactions.
185             axi_awready <= '1';
186             elsif (axs_s0_AXI_BREADY = '1' and axi_bvalid = '1') then
187                 aw_en <= '1';
188                 axi_awready <= '0';
189           else
190             axi_awready <= '0';
191           end if;
192         end if;
193       end if;
194     end process;
195 
196 
197     -- Implement axi_awaddr latching
198     -- This process is used to latch the address when both
199     -- S_AXI_AWVALID and S_AXI_WVALID are valid.
```

```
200
201
202     process (clk)
203     begin
204       if rising_edge(clk) then
205         if reset_n = '0' then
206           axi_awaddr <= (others => '0');
207         else
208           if (axi_awready = '0' and axs_s0_AXI_AWVALID = '1' and axs_s0_AXI_WVALID
                  = '1' and aw_en = '1') then
209             -- Write Address latching
210             axi_awaddr <= axs_s0_AXI_AWADDR;
211           end if;
212         end if;
213       end if;
214     end process;
215
216
217     -- Implement axi_wready generation
218     -- axi_wready is asserted for one S_AXI_ACLK clock cycle when both
219     -- S_AXI_AWVALID and S_AXI_WVALID are asserted. axi_wready is
220     -- de-asserted when reset is low.
221
222
223     process (clk)
224     begin
225       if rising_edge(clk) then
226         if reset_n = '0' then
227           axi_wready <= '0';
228         else
229           if (axi_wready = '0' and axs_s0_AXI_WVALID = '1' and axs_s0_AXI_AWVALID
                  = '1' and aw_en = '1') then
230             -- slave is ready to accept write data when
231             -- there is a valid write address and write data
232             -- on the write address and data bus. This design
233             -- expects no outstanding transactions.
234             axi_wready <= '1';
235           else
236             axi_wready <= '0';
237           end if;
238         end if;
239       end if;
240     end process;
241
242
243     -- Implement memory mapped register select and write logic generation
244     -- The write data is accepted and written to memory mapped registers when
245     -- axi_awready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are asserted. Write
          strobes are used to
246     -- select byte enables of slave registers while writing.
247     -- These registers are cleared when reset (active low) is applied.
248     -- Slave register write enable is asserted when valid address and data are
          available
249     -- and the slave is ready to accept the write address and write data.
250     slv_reg_wren <= axi_wready and axs_s0_AXI_WVALID and axi_awready and
          axs_s0_AXI_AWVALID ;
251
252
253     process (clk)
254     variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
255     begin
256       if rising_edge(clk) then
257         if reset_n = '0' then
258           slv_reg0 <= (others => '0');
```

```
259         slv_reg1 <= (others => '0');
260         slv_reg2 <= (others => '0');
261         slv_reg3 <= (others => '0');
262       else
263         loc_addr := axi_awaddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);
264         if (slv_reg_wren = '1') then
265           case loc_addr is
266             when b"00" =>
267               for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
268                 if ( axs_s0_AXI_WSTRB(byte_index) = '1' ) then
269                   -- Respective byte enables are asserted as per write strobes
270                   -- slave registor 0
271                   slv_reg0(byte_index*8+7 downto byte_index*8) <=
                          axs_s0_AXI_WDATA(byte_index*8+7 downto byte_index*8);
272                 end if;
273               end loop;
274             when b"01" =>
275               for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
276                 if ( axs_s0_AXI_WSTRB(byte_index) = '1' ) then
277                   -- Respective byte enables are asserted as per write strobes
278                   -- slave registor 1
279                   slv_reg1(byte_index*8+7 downto byte_index*8) <=
                          axs_s0_AXI_WDATA(byte_index*8+7 downto byte_index*8);
280                 end if;
281               end loop;
282             when b"10" =>
283               for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
284                 if ( axs_s0_AXI_WSTRB(byte_index) = '1' ) then
285                   -- Respective byte enables are asserted as per write strobes
286                   -- slave registor 2
287                   slv_reg2(byte_index*8+7 downto byte_index*8) <=
                          axs_s0_AXI_WDATA(byte_index*8+7 downto byte_index*8);
288                 end if;
289               end loop;
290             when b"11" =>
291               for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
292                 if ( axs_s0_AXI_WSTRB(byte_index) = '1' ) then
293                   -- Respective byte enables are asserted as per write strobes
294                   -- slave registor 3
295                   slv_reg3(byte_index*8+7 downto byte_index*8) <=
                          axs_s0_AXI_WDATA(byte_index*8+7 downto byte_index*8);
296                 end if;
297               end loop;
298             when others =>
299               slv_reg0 <= slv_reg0;
300               slv_reg1 <= slv_reg1;
301               slv_reg2 <= slv_reg2;
302               slv_reg3 <= slv_reg3;
303           end case;
304         end if;
305       end if;
306     end if;
307   end process;
308
309
310   -- Implement write response logic generation
311   -- The write response and response valid signals are asserted by the slave
312   -- when axi_wready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are asserted.
313   -- This marks the acceptance of address and indicates the status of
314   -- write transaction.
315
316
317   process (clk)
318   begin
```

```
319       if rising_edge(clk) then
320         if reset_n = '0' then
321           axi_bvalid  <= '0';
322           axi_bresp   <= "00"; --need to work more on the responses
323         else
324           if (axi_awready = '1' and axs_s0_AXI_AWVALID = '1' and axi_wready = '1'
                  and axs_s0_AXI_WVALID = '1' and axi_bvalid = '0'  ) then
325             axi_bvalid <= '1';
326             axi_bresp  <= "00";
327           elsif (axs_s0_AXI_BREADY = '1' and axi_bvalid = '1') then   --check if
                  bready is asserted while bvalid is high)
328             axi_bvalid <= '0';                                        -- (there is a
                   possibility that bready is always asserted high)
329           end if;
330         end if;
331       end if;
332     end process;
333
334
335     -- Implement axi_arready generation
336     -- axi_arready is asserted for one S_AXI_ACLK clock cycle when
337     -- S_AXI_ARVALID is asserted. axi_awready is
338     -- de-asserted when reset (active low) is asserted.
339     -- The read address is also latched when S_AXI_ARVALID is
340     -- asserted. axi_araddr is reset to zero on reset assertion.
341
342
343     process (clk)
344     begin
345       if rising_edge(clk) then
346         if reset_n = '0' then
347           axi_arready <= '0';
348           axi_araddr  <= (others => '1');
349         else
350           if (axi_arready = '0' and axs_s0_AXI_ARVALID = '1') then
351             -- indicates that the slave has acceped the valid read address
352             axi_arready <= '1';
353             -- Read Address latching
354             axi_araddr  <= axs_s0_AXI_ARADDR;
355           else
356             axi_arready <= '0';
357           end if;
358         end if;
359       end if;
360     end process;
361
362
363     -- Implement axi_arvalid generation
364     -- axi_rvalid is asserted for one S_AXI_ACLK clock cycle when both
365     -- S_AXI_ARVALID and axi_arready are asserted. The slave registers
366     -- data are available on the axi_rdata bus at this instance. The
367     -- assertion of axi_rvalid marks the validity of read data on the
368     -- bus and axi_rresp indicates the status of read transaction.axi_rvalid
369     -- is deasserted on reset (active low). axi_rresp and axi_rdata are
370     -- cleared to zero on reset (active low).
371     process (clk)
372     begin
373       if rising_edge(clk) then
374         if reset_n = '0' then
375           axi_rvalid <= '0';
376           axi_rresp  <= "00";
377         else
378           if (axi_arready = '1' and axs_s0_AXI_ARVALID = '1' and axi_rvalid = '0')
                  then
```

```
379             -- Valid read data is available at the read data bus
380             axi_rvalid <= '1';
381             axi_rresp  <= "00"; -- 'OKAY' response
382           elsif (axi_rvalid = '1' and axs_s0_AXI_RREADY = '1') then
383             -- Read data is accepted by the master
384             axi_rvalid <= '0';
385           end if;
386         end if;
387       end if;
388     end process;
389
390
391     -- Implement memory mapped register select and read logic generation
392     -- Slave register read enable is asserted when valid address is available
393     -- and the slave is ready to accept the read address.
394     slv_reg_rden <= axi_arready and axs_s0_AXI_ARVALID and (not axi_rvalid) ;
395
396
397     process (slv_reg0, slv_reg1, slv_reg2, slv_reg3, axi_araddr, reset_n,
            slv_reg_rden, signal_result, signal_busy)
398     variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
399     begin
400         -- Address decoding for reading registers
401         loc_addr := axi_araddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);
402         case loc_addr is
403           when b"00" =>
404             reg_data_out <= slv_reg0;
405           when b"01" =>
406             reg_data_out <= slv_reg1;
407           when b"10" =>
408             reg_data_out <= signal_result; -- read the result of multiplication
                    from the booth multiplier
409           when b"11" =>
410             reg_data_out <= slv_reg3(31 downto 2)&signal_busy&slv_reg3(0); -- read
                     the busy signal from the booth multiplier in bit 1 of the
                    register, and bit 0 is used for start signal
411           when others =>
412             reg_data_out  <= (others => '0');
413         end case;
414     end process;
415
416
417     -- Output register or memory read data
418     process( clk ) is
419     begin
420       if (rising_edge (clk)) then
421         if ( reset_n = '0' ) then
422           axi_rdata  <= (others => '0');
423         else
424           if (slv_reg_rden = '1') then
425             -- When there is a valid read address (S_AXI_ARVALID) with
426             -- acceptance of read address by the slave (axi_arready),
427             -- output the read dada
428             -- Read address mux
429               axi_rdata <= reg_data_out;     -- register read data
430           end if;
431         end if;
432       end if;
433     end process;
434
435
436
437
438     -- Booth multiplier port map and logic
```

```
439
440     booth_instance: booth_mult
441     port map(
442             clk     => clk,
443             resetn  => reset_n,
444             start       => slv_reg3(0),
445             busy        => signal_busy,
446             irq     => irq,
447             ack     => slv_reg3(2),
448             data_a      => slv_reg0(15 downto 0),
449             data_b      => slv_reg1(15 downto 0),
450             result      => signal_result
451         );
452     slv_reg3(1) <= signal_busy;
453     slv_reg2 <= signal_result;
454
455     -- Booth multiplier logic ends
456
457
458 end arch_imp;
```

Listing 1: AXI4-Lite (`axi4_lite_booth.vhd`).

3 Software Application

3.1 Booth ISR

The ISR is triggered when the multiplier raises `irq`. It clears the interrupt at the source by toggling the `ack` bit (bit 2 of slv_reg3) and sets a multiplication flag.

```
1  #include "address_map_arm.h"
2  #include <stdio.h>
3  extern volatile int MULTIPLICATION_FINISHED;
4
5
6  /*
       ********************************************************************************
7  * Booth IRQ - Interrupt Service Routine
8  *
9  * Since, IRQ is generated by the booth multiplier, we need to clear the interrupt
       by toggling the ack bit (bit 2) in slv_reg3.
10 * After clearing the interrupt, we set the MULTIPLICATION_FINISHED flag to 1.
11 ********************************************************************************
       */
12 void booth_ISR(void) {
13     volatile int *status = (int *)0xFF20000C;  // reads the data at slv_reg3 base
           address
14
15     printf("Checking the Booth ISR\n");
16     int val = *status; // temporarily store the value of slv_reg3 in a variable
17
18     // Enable ack
19     val |= (1 << 2);       // ack = 1
20     *status = val;
21
22     // Disable ack
23     val = *status;
24     val &= ~(1 << 2);      // ack = 0
25     *status = val;
26
27     // Post-multiplication flag
```

```
    MULTIPLICATION_FINISHED = 1;
    printf("Exiting the Booth ISR\n");

    return;
}
```

Listing 2: Interrupt service routine (`booth_ISR.c`).

3.2 Exception Handler

We reused the same Exceptions.c file that was originally used in the lab2 counter. The only change has been done where we called the booth ISR(). We do not use IRQ enable; so no external keys where configured.

```
#include "address_map_arm.h"
#include "defines.h"
#include "interrupt_ID.h"
#include <stdio.h>

void booth_ISR(void);

void __attribute__((interrupt)) __cs3_isr_irq(void)
{
    int address = MPCORE_GIC_CPUIF + ICCIAR;
    int int_ID  = *((int *)address);

    if (int_ID == KEYS_IRQ) {
        printf("Keys IRQ\n");
    } else {
        while (1) { ; }
    }

    address = MPCORE_GIC_CPUIF + ICCEOIR;
    *((int *)address) = int_ID;
    return;
}

void __attribute__((interrupt)) __cs3_reset    (void) { while (1) { ; } }
void __attribute__((interrupt)) __cs3_isr_undef(void) { while (1) { ; } }
void __attribute__((interrupt)) __cs3_isr_swi  (void) { while (1) { ; } }
void __attribute__((interrupt)) __cs3_isr_pabort(void){ while (1) { ; } }
void __attribute__((interrupt)) __cs3_isr_dabort(void){ while (1) { ; } }
void __attribute__((interrupt)) __cs3_isr_fiq  (void) { while (1) { ; } }

void set_A9_IRQ_stack(void)
{
    int stack = A9_ONCHIP_END - 7;
    int mode  = INT_DISABLE | IRQ_MODE;
    asm("msr cpsr, %[ps]" : : [ps] "r"(mode));
    asm("mov sp,   %[ps]" : : [ps] "r"(stack));
    mode = INT_DISABLE | SVC_MODE;
    asm("msr cpsr, %[ps]" : : [ps] "r"(mode));
}

void enable_A9_interrupts(void)
{
    int status = SVC_MODE | INT_ENABLE;
    asm("msr cpsr, %[ps]" : : [ps] "r"(status));
}

void config_GIC(void)
{
    int address;
```

```
    *((int *)0xFFFED848) = 0x00000100;    // SPI 73 -> CPU0
    *((int *)0xFFFED108) = 0x00000F00;    // enable SPI 72-75

    address = MPCORE_GIC_CPUIF + ICCPMR;
    *((int *)address) = 0xFFFF;

    address = MPCORE_GIC_CPUIF + ICCICR;
    *((int *)address) = ENABLE;

    address = MPCORE_GIC_DIST + ICDDCR;
    *((int *)address) = ENABLE;
}
```

Listing 3: Exception handlers and GIC setup (`exceptions.c`).

3.3 Main Application

It starts to read two operands from the JTAG-UART, push them to slv_reg0 and slv_reg1, initialize `start`, looks for IRQ clears it if set and preforms a multiplication and stores the result and prints.

```
#include "address_map_arm.h"
#include <stdio.h>
#include <stdlib.h>


// Set the base addresses of the registers for the Booth multiplier
#define REG0 0xFF200000 // multiplicand data_a
#define REG1 0xFF200004 // multiplier data_b
#define REG2 0xFF200008 // stores the result of multiplication
#define REG3 0xFF20000C // start/busy/ack status


void set_A9_IRQ_stack(void);
void config_GIC(void);
void enable_A9_interrupts(void);


volatile int MULTIPLICATION_FINISHED = 0;


int main(void)
{
 //Reading the base address of the registers for the Booth multiplier, and
     storing them in pointers.
 volatile int *reg0 = (int *)REG0;
 volatile int *reg1 = (int *)REG1;
 volatile int *reg2 = (int *)REG2;
 volatile int *reg3 = (int *)REG3;
 int result;


 set_A9_IRQ_stack();       // initialize the stack pointer for IRQ mode
 config_GIC();             // configure the general interrupt controller
 enable_A9_interrupts(); // enable interrupts


 printf("\n Radix-4 Booth Multiplier \n");


 while (1)
 {
```

```
    // Check if the multiplier is busy before starting a new multiplication operation.
    printf("\n 1: Checking busy bit\n");
    int busy = (*reg3 >> 1) & 0x1;
    printf("Busy: %d\n", busy);


    if (busy == 1)
    {
      //Wait for the busy bit to disable before
      //the next operation
      printf("Multiplier is busy\n");
      continue;
    }


    // If IRQ is generated by the booth multiplier, we need to clear the interrupt by toggling the ack bit
    printf("\n 2: Clear IRQ\n");
    int val = *reg3;
    val |= (1 << 2); // ack = 1
    *reg3 = val; // Write to volatile register
    val = *reg3;
    val &= ~(1 << 2); // ack = 0
    *reg3 = val; // Write to volatile register


    //  Read user input for multiplicand and multiplier, and store them in the corresponding registers
    int16_t multiplicand_input, multiplier_input; //16 bit input for the user
    printf("Enter data_a (16-bit): ");
    scanf("%hd", &multiplicand_input);

    printf("Enter data_b (16-bit): ");
    scanf("%hd", &multiplier_input);


    // Storing the user input in the corresponding registers
    *reg0 = multiplicand_input; //Write data_a to slv_reg0
    *reg1 = multiplier_input; //Write data_b to slv_reg1
    printf("Wrote %d (%d) to REG0, %d (%d) to REG1, \n", multiplicand_input, *reg0, multiplier_input, *reg1);




    // Set the start bit to 1 to start the multiplication operation
    val = *reg3; //Read status of start/busy/ack from slv_reg3
    val |= (1 << 0); // set start = 1
    *reg3 = val; //Write to reg3
    printf("Started multiplication\n");

    // Wait for the multiplication to finish by monitoring the MULTIPLICATION_FINISHED flag
    printf("Waiting for interrupt...\n");
    while (!MULTIPLICATION_FINISHED)
    {
    }
    MULTIPLICATION_FINISHED = 0; //Clear flag

    printf("Interrupt received!\n");

    // Clear the start bit to 0 after acknowledging the completion of the multiplication operation
```

```
    val = *reg3;
    val &= ~(1 << 0); // start = 0
    *reg3 = val;

    // Read the result from the corresponding register and print it
    printf("Result: %d\n", *reg2);


 }


 return 0;
}
```

Listing 4: Main application (`lab2.c`).

4 Results

We have used functional testing to verify our system with the different combinations of signed values mentioned in table 2. The software was complied successfully and was loaded in Intel-FPGA-monitor-program to check the following values in the terminal. The terminal can be used to check the data being stored in the registers just to confirm if the registers have been mapped correctly.

Screenshots illustrating the design behavior for all the stimuli applied are provided below in the terminal 1.

Table 2: Sample results obtained from the running board.

data_a	data_b	Reported result
12	5	60
-50	-15	750
-8	20	-160
30	-250	-7500

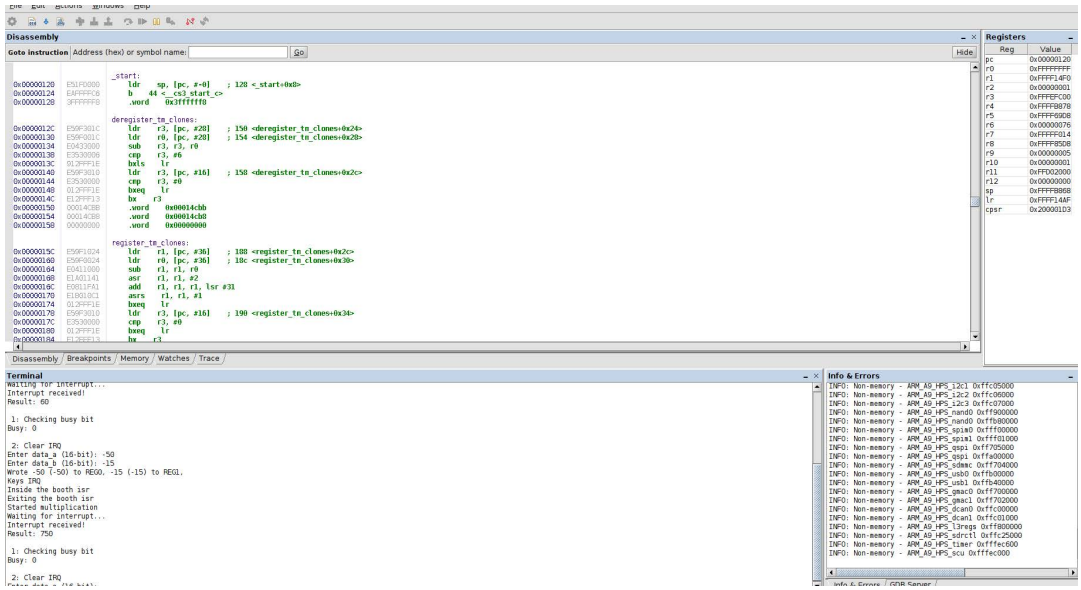


Figure 1: Output terminal Intel FPGA Monitor program for single case

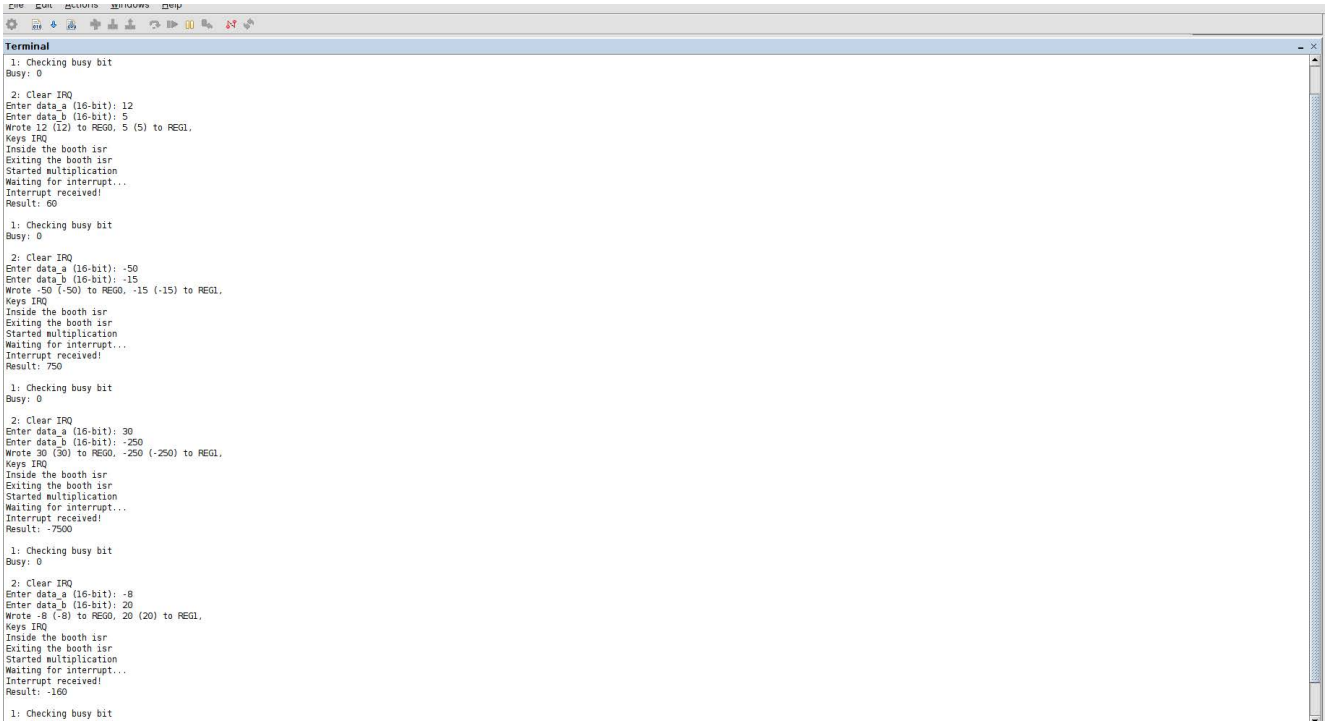


Figure 2: Output terminal Intel FPGA Monitor program for all cases (cases for table)

5 Resource Utilization

The design was successfully synthesized and analyzed using Quartus Prime. Table 3 summarizes the hardware resource utilization for the implemented system targeting the Cyclone V FPGA family. A total of 973 registers were implemented using the logic used.

Table 3: Resource Utilization Summary

Parameter	**Value**
Analysis & Synthesis Status	Successful
Compilation Date	Mon May 18 19:49:10 2026
Quartus Prime Version	21.1.0 Build 842 Standard Edition
Revision Name	`lab1`
Top-Level Entity	`lab1`
FPGA Family	Cyclone V
Total Registers	973
Total Pins	88
Total Virtual Pins	0
Total Block Memory Bits	1,024
Total DSP Blocks	0
Total PLLs	1
Total DLLs	1

6 Timing Analysis Report Summary

The timing report indicates that the design satisfies the timing requirements under all tested operating conditions, as all slack values are positive. Additionally, the Total Negative Slack (TNS) is zero in every case, confirming that no timing violations are present in the system.

Table 4: Timing Analysis Summary

Model	**Voltage**	**Temperature**	**Timing Check**	**Slack / TNS (ns)**
Slow	1100mV	85°C	Setup	1.730 / 0.000
Slow	1100mV	85°C	Hold	0.143 / 0.000
Slow	1100mV	85°C	Recovery	3.375 / 0.000
Slow	1100mV	85°C	Removal	0.470 / 0.000
Slow	1100mV	85°C	Minimum Pulse Width	0.608 / 0.000
Slow	1100mV	0°C	Setup	1.727 / 0.000
Slow	1100mV	0°C	Hold	0.162 / 0.000
Slow	1100mV	0°C	Recovery	3.468 / 0.000
Slow	1100mV	0°C	Removal	0.460 / 0.000
Slow	1100mV	0°C	Minimum Pulse Width	0.613 / 0.000
Fast	1100mV	85°C	Setup	2.110 / 0.000
Fast	1100mV	85°C	Hold	0.077 / 0.000
Fast	1100mV	85°C	Recovery	3.727 / 0.000
Fast	1100mV	85°C	Removal	0.466 / 0.000
Fast	1100mV	85°C	Minimum Pulse Width	0.883 / 0.000
Fast	1100mV	0°C	Setup	2.110 / 0.000
Fast	1100mV	0°C	Hold	0.058 / 0.000
Fast	1100mV	0°C	Recovery	3.825 / 0.000
Fast	1100mV	0°C	Removal	0.406 / 0.000
Fast	1100mV	0°C	Minimum Pulse Width	0.887 / 0.000